

Understanding Deployments

When you're trying to troubleshoot a particular deployment attempt, it's helpful to have a clear understanding of how a typical deployment works. *Deployment* refers to the process of taking a project that works on your local system, and copying that project to a remote server in a way that allows it to respond to requests from any user on the internet. The remote environment differs from a typical local system in a number of important ways: it's probably not the same operating system (OS) as the one you're using, and it's most likely one of many virtual servers on a single physical server.

When you deploy a project, or *push* it to the remote server, the following steps need to be taken:

- Create a virtual server on a physical machine at a datacenter.
- Establish a connection between the local system and the remote server.
- Copy the project's code to the remote server.
- Identify all of the project's dependencies and install them on the remote server.
- Set up a database and run any existing migrations.
- Copy static files (CSS, JavaScript files, and media files) to a place where they can be served efficiently.
- Start a server to handle incoming requests.
- Start routing incoming requests to the project, once it's ready to handle requests.

When you consider all that goes into a deployment, it's no wonder deployments often fail. Fortunately, once you gain an understanding of what should be happening, you'll stand a better chance of identifying what went wrong. If you can identify what went wrong, you might be able to identify a fix that will make the next deployment attempt successful.

You can develop locally on one kind of OS and push to a server running a different OS. It's important to know what kind of system you're pushing to, because that can inform some of your troubleshooting work. At the time of this writing, a basic remote server on Platform.sh runs Debian Linux; most remote servers are Linux-based systems.

Basic Troubleshooting

Some troubleshooting steps are specific to each OS, but we'll get to that in a moment. First, let's consider the steps everyone should try when troubleshooting a deployment.

Your best resource is the output generated during the attempted push. This output can look intimidating; if you're new to deploying apps, it can look highly technical, and there's usually a lot of it. The good news is you don't need to understand everything in the output. You should have two goals when skimming log output: identify any deployment steps that worked, and identify any steps that didn't. If you can do this, you might be